Machine Learning Models Guide

A Visual Reference to Six Fundamental Classification Algorithms

Linear Logistic Regression · Linear SVM · K Nearest Neighbours

Contents

01 Introduction

02 Linear Logistic Regression

03 Linear Support Vector Machine

04 K Nearest Neighbours

05 Naive Bayes

06 Decision Tree

07 Random Forest

08 Model Comparison

Introduction

This guide provides a structured reference to six fundamental machine learning algorithms for classification tasks. Each model is explained with its core intuition, mathematical formulation, key characteristics, and a visual diagram. Understanding these classical methods is essential for any practitioner, as they form the building blocks upon which more complex techniques are built.

Classification is the task of predicting a discrete class label for a given input. The algorithms covered in this document represent different philosophical approaches to this problem: from probabilistic modelling and geometric margin maximization, to instance-based learning, Bayesian inference, and recursive partitioning.

Overview of Models

Model	Family	Learning Type	Decision Boundary
Logistic Regression	Generalized Linear Model	Eager	Linear (via sigmoid)
Linear SVM	Maximum Margin	Eager	Linear (max margin)
K Nearest Neighbours	Instance-based	Lazy	Non-linear (piecewise)
Naive Bayes	Bayesian / Generative	Eager	Non-linear
Decision Tree	Tree-based	Eager	Axis-aligned splits
Random Forest	Ensemble (Bagging)	Eager	Non-linear ensemble

Linear Logistic Regression

Core Intuition

Logistic Regression models the probability that an input belongs to a particular class. Despite its name, it is a classification algorithm, not a regression one. It applies the **sigmoid function** to a linear combination of features, squeezing the output into the range [0, 1]. This S-shaped curve provides a smooth, interpretable mapping from feature space to class probabilities.

The model learns a set of weights and a bias term during training. A threshold (typically 0.5) converts the probability into a discrete class prediction. Values above the threshold are assigned to the positive class; values below go to the negative class.

Mathematical Formulation

$$p(y = 1 \mid x) = \sigma(w \cdot x + b) = 1 / (1 + e^{-(w \cdot x + b)})$$

Where σ denotes the sigmoid function, **w** is the weight vector, **x** is the input feature vector, and **b** is the bias term. Training minimizes the cross-entropy loss function:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Visual Diagram



Figure 1: The sigmoid function maps any real-valued input to a probability between 0 and 1. The decision boundary at p = 0.5 separates the two classes.

Key Characteristics

STRENGTHS

- Outputs well-calibrated class probabilities
- Fast to train and predict, even on large datasets
- Highly interpretable through feature weights
- Low variance makes it less prone to overfitting
- Regularization (L1/L2) helps prevent overfitting

LIMITATIONS

- Assumes a linear decision boundary between classes
- Cannot naturally model non-linear relationships
- Sensitive to outliers in the training data
- Requires careful feature scaling for optimal performance
- Assumes independence of errors (no multicollinearity)

Linear Support Vector Machine

Core Intuition

A Support Vector Machine (SVM) finds the **optimal hyperplane** that separates two classes while maximizing the margin between them. The margin is defined as the distance between the hyperplane and the closest training points from each class. These closest points are called **support vectors** and are the only ones that influence the decision boundary.

By maximizing the margin, SVM achieves better generalization than models that simply achieve zero training error. A wider margin means the classifier is more robust to small perturbations in the input data.

Mathematical Formulation

$$\text{Minimize: } \frac{1}{2}\|w\|^2 \quad \text{Subject to: } y_i(w \cdot x_i + b) \geq 1$$

This is a convex optimization problem. The constraint ensures all points are correctly classified and lie on the correct side of the margin. For non-separable data, a soft-margin formulation introduces slack variables ξ_i that allow some misclassifications while penalizing them:

$$\text{Minimize: } \frac{1}{2}\|w\|^2 + C \sum \xi_i$$

The parameter **C** controls the trade-off between maximizing the margin and minimizing classification errors.

Visual Diagram

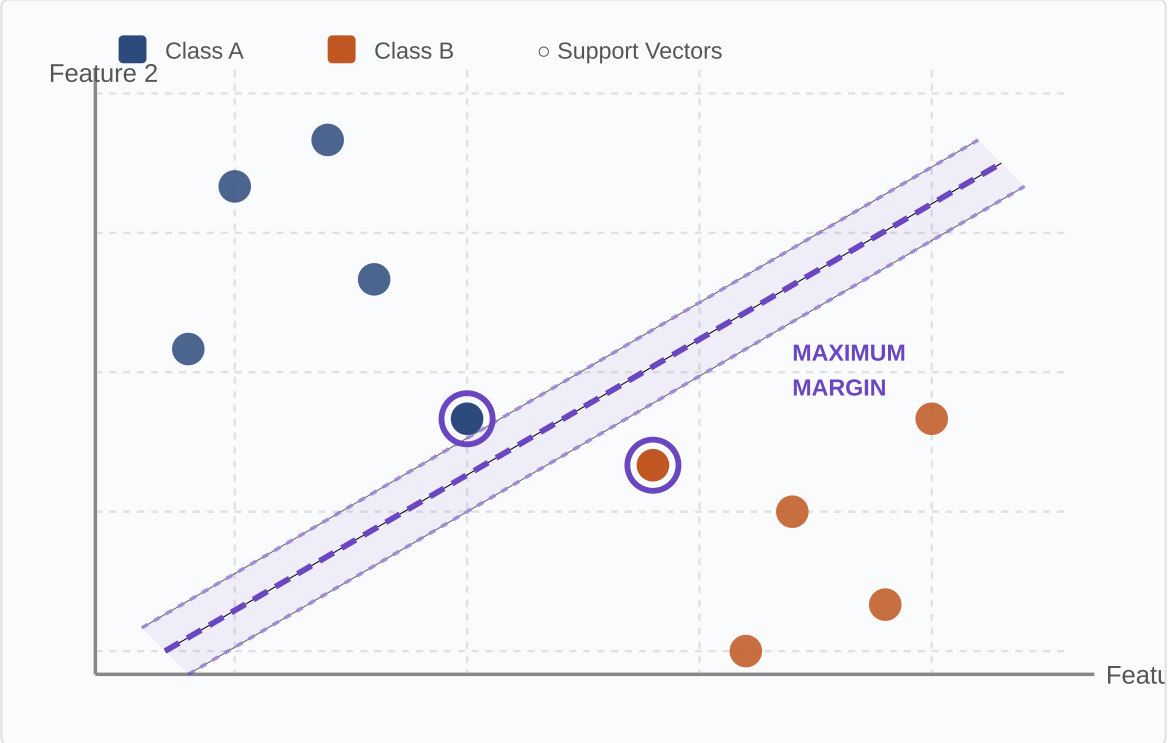


Figure 2: The SVM finds the hyperplane that maximizes the margin between classes. Only the circled support vectors determine the boundary; other points can be removed without changing it.

Key Characteristics

STRENGTHS

- Maximum margin principle provides strong generalization
- Works effectively in high-dimensional feature spaces
- Memory efficient — only support vectors are needed
- Versatile via kernel functions for non-linear problems
- Unique global optimum (convex optimization)

LIMITATIONS

- Does not natively output probability estimates
- Training can be slow on very large datasets
- Sensitive to noise and mislabeled examples
- Requires careful tuning of the C parameter
- Feature scaling is important for good performance

K Nearest Neighbours

Core Intuition

K Nearest Neighbours (KNN) is a **lazy learner** — it does not train an explicit model. Instead, it simply stores the entire training dataset. When a prediction is needed for a new point, it finds the K closest training examples and lets them vote. The majority class among these neighbours becomes the prediction.

The choice of K is critical. A small K makes the model sensitive to noise and local fluctuations (risk of overfitting), while a large K smooths out the decision boundary but may blur important distinctions (risk of underfitting). Common distance metrics include Euclidean, Manhattan, and Minkowski distances.

Mathematical Formulation

$$d(x, x_i) = \sqrt{\Sigma_{j=1}^{n} (x_j - x_{ij})^2} \quad \text{(Euclidean distance)}$$

For classification, the predicted class is the mode of the K nearest neighbours:

$$\hat{y} = \text{mode}\{ y_i \mid x_i \in \text{K-nearest}(x) \}$$

Visual Diagram

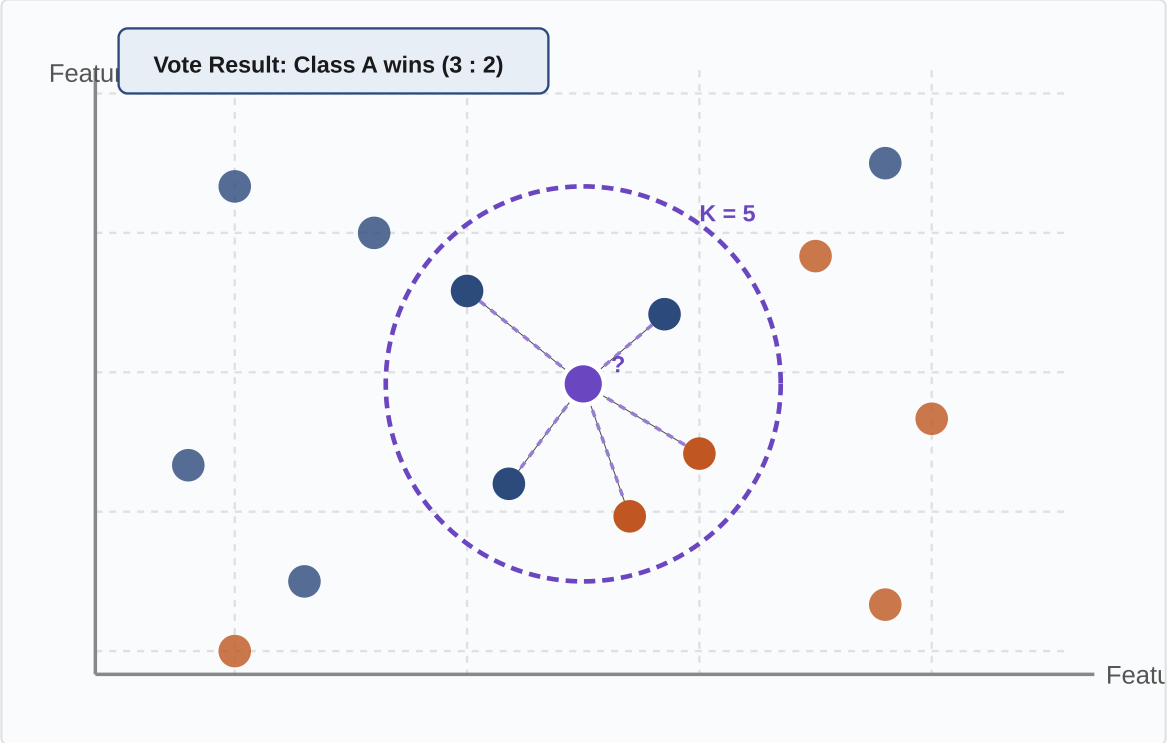


Figure 3: KNN with K=5. The query point (purple) examines its 5 closest neighbours. Three belong to Class A and two to Class B, so the prediction is Class A.

Key Characteristics

STRENGTHS

- No training phase — extremely fast "learning"
- Simple to understand, implement, and explain
- Naturally adapts as new training data arrives
- Handles multi-class problems without modification
- No assumptions about data distribution (non-parametric)

LIMITATIONS

- Prediction is slow on large datasets (O(n) per query)
- Very sensitive to feature scaling and irrelevant features
- Performance degrades in high dimensions (curse of dim.)
- Requires storing the entire training dataset
- Choice of distance metric and K value is critical

Naive Bayes

Core Intuition

Naive Bayes applies **Bayes' theorem** with a simplifying "naive" assumption: all features are conditionally independent given the class. While this assumption rarely holds in practice, the resulting model is surprisingly effective and computationally efficient.

Two common variants are used depending on feature type: **Gaussian Naive Bayes** assumes continuous features follow a normal distribution, while **Categorical Naive Bayes** models discrete features using categorical distributions. The algorithm is particularly popular for text classification tasks such as spam detection and sentiment analysis.

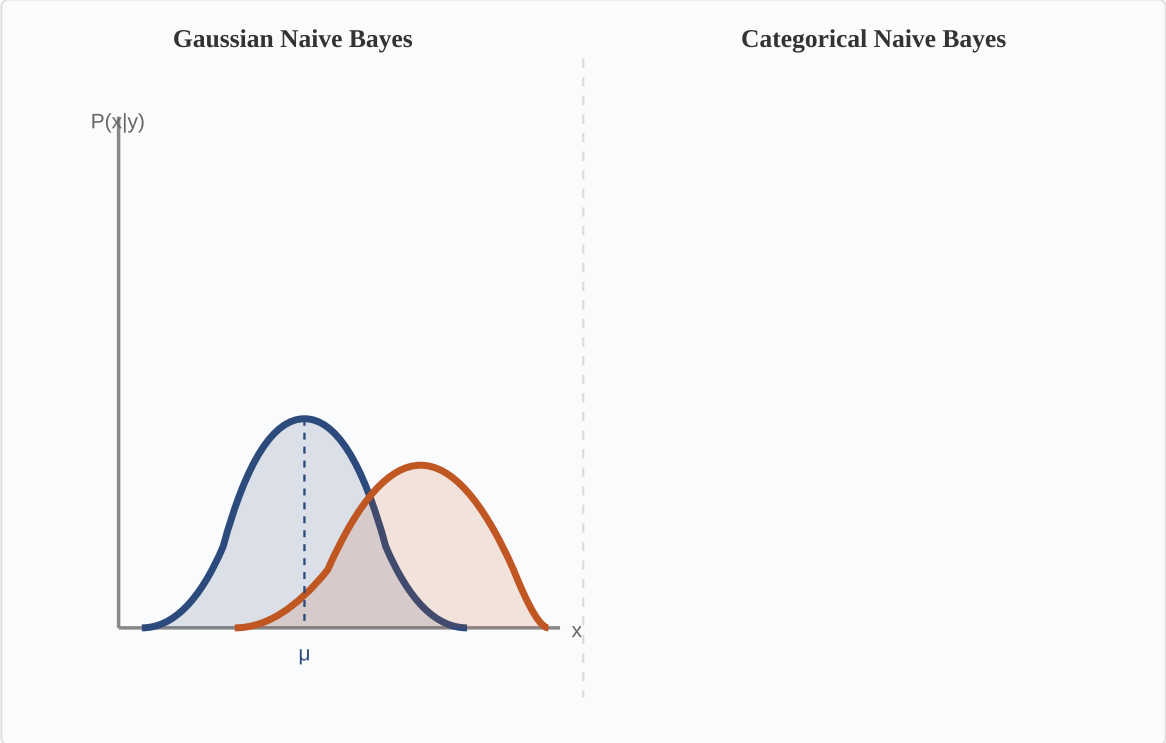
Mathematical Formulation

$$P(y \mid x) = P(y) \cdot \Pi_{i=1}^{n} P(x_i \mid y) \;/\; P(x)$$

Since P(x) is constant for all classes, we only need to maximize the numerator. For Gaussian NB, each feature likelihood is:

$$P(x_i \mid y) = 1 \;/\; (\sqrt{2\pi\sigma_y^2}) \cdot \exp(-(x_i - \mu_y)^2 \;/\; (2\sigma_y^2))$$

Visual Diagram



$_{A}\mu_{B}$P(x|y=A) P(x|y=B) Value Freq red blue green red blue green Class A Class B 0 5 7

Figure 4: Left — Gaussian NB fits a normal distribution per class per feature. Right — Categorical NB counts feature occurrences per class (shown here for a "color" feature with values red, blue, green).

Key Characteristics

STRENGTHS

- Extremely fast training and prediction
- Works well even with high-dimensional data
- Requires very little training data to work well
- Excellent baseline for text classification tasks
- Naturally handles multi-class problems

LIMITATIONS

- Feature independence assumption is rarely true
- Can be a poor probability estimator
- Zero-frequency problem needs smoothing (Laplace)
- Gaussian variant assumes normality of features
- Not ideal when features are strongly correlated

Decision Tree

Core Intuition

A Decision Tree learns a hierarchy of **if-then-else rules** that recursively partition the feature space. Each internal node tests a single feature against a threshold, each branch represents the outcome of the test, and each leaf node provides a class prediction. The tree is built top-down by choosing at each step the split that best separates the classes.

The quality of a split is measured using impurity metrics such as **Gini impurity** or **entropy**. The goal is to maximize information gain — the reduction in impurity achieved by the split. Without constraints, decision trees tend to grow deep and overfit; regularization techniques like limiting maximum depth or requiring minimum samples per leaf help prevent this.

Mathematical Formulation

$$\text{Gini} = 1 - \sum_{i=1}^{C} p_i^2 \quad | \quad \text{Entropy} = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

Information gain measures how much impurity decreases after a split:

$$IG = I(\text{parent}) - \left(n_{\text{left}}/n\right) I(\text{left}) - \left(n_{\text{right}}/n\right) I(\text{right})$$

Visual Diagram

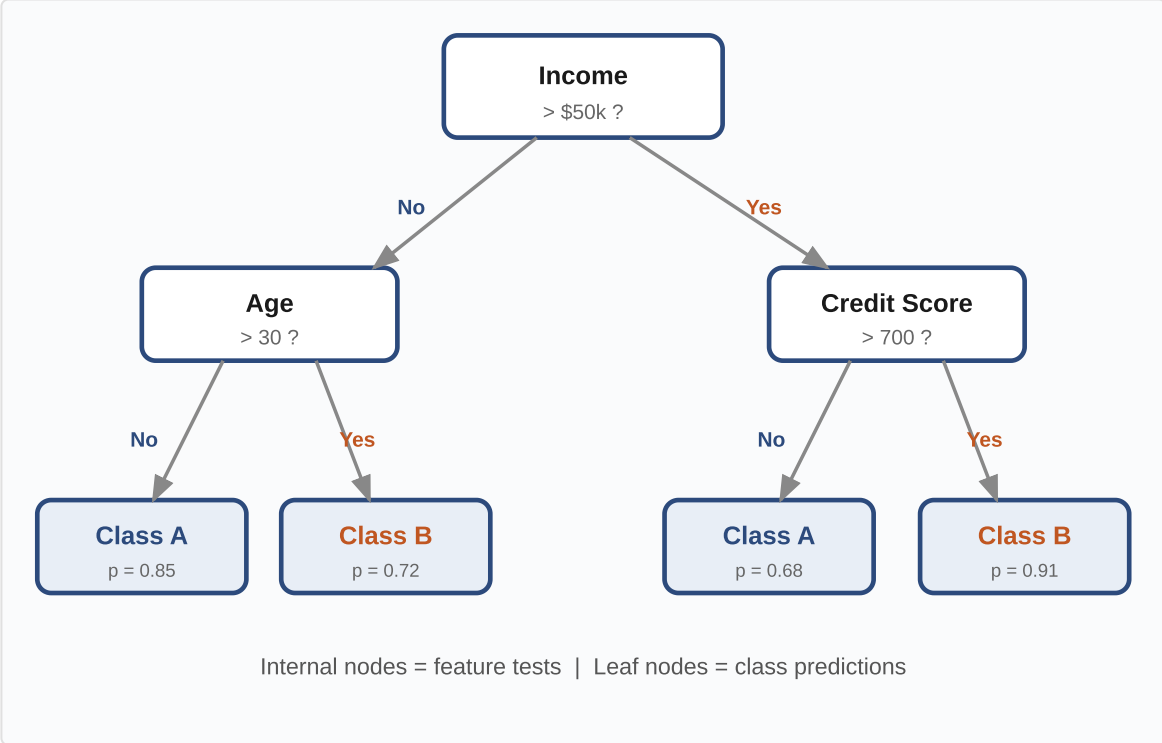


Figure 5: A decision tree for credit approval. Starting from the root, each node asks a binary question about a feature. Following the branches leads to a leaf node that provides the final prediction with confidence.

Key Characteristics

STRENGTHS

- Highly interpretable — can be visualized as rules
- Requires little or no data preprocessing/scaling
- Naturally handles both numerical and categorical data
- Captures non-linear relationships and interactions
- White-box model — decisions are explainable

LIMITATIONS

- Easily overfits to noise without pruning/regularization
- High variance — small data changes alter tree structure
- Biased toward features with many levels (categories)
- Cannot model smooth decision boundaries well
- Greedy splits may not yield globally optimal tree

Random Forest

Core Intuition

A Random Forest is an **ensemble method** that trains many decision trees and combines their predictions. Each tree is trained on a random bootstrap sample of the data (bagging), and at each split only a random subset of features is considered. These two sources of randomness de-correlate the trees, allowing the ensemble to achieve better generalization than any individual tree.

For classification, the final prediction is the **majority vote** across all trees. Random Forests reduce the high variance that plagues individual decision trees while maintaining their ability to model complex patterns. They are widely regarded as one of the most reliable "off-the-shelf" algorithms.

Mathematical Formulation

$$\hat{y}(x) = \text{mode}\{T_1(x), T_2(x), \ldots, T_N(x)\}$$

Where each T_i is a decision tree trained on a bootstrap sample with random feature selection. The out-of-bag (OOB) samples — those not included in a tree's bootstrap — can be used to estimate generalization performance without a separate validation set.

Visual Diagram

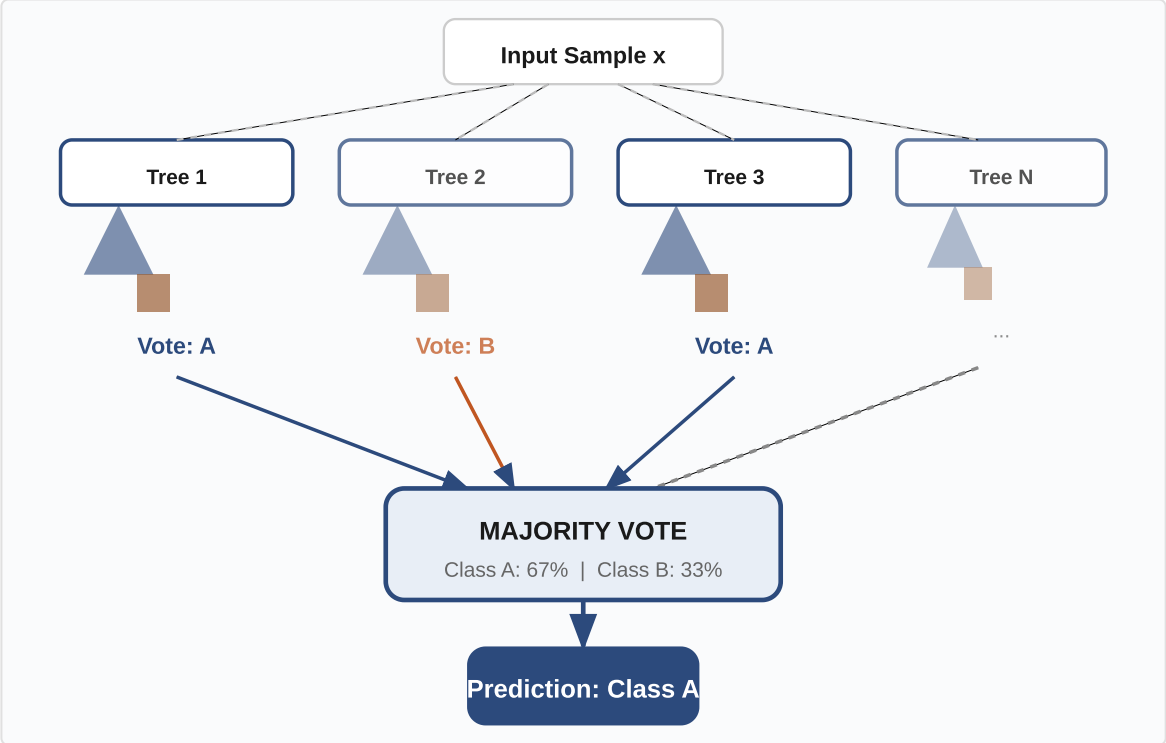


Figure 6: A Random Forest ensemble. Each tree is trained on different data and sees different features. Trees vote independently, and the majority class becomes the final prediction.

Key Characteristics

STRENGTHS

- Reduces overfitting through ensemble averaging
- Robust to noise, outliers, and class imbalance
- Provides built-in feature importance estimates
- Often top-tier performance with minimal tuning
- Handles mixed feature types and missing values

LIMITATIONS

- Less interpretable than a single decision tree
- Prediction speed scales with number of trees
- Memory intensive — stores many full trees
- Can still overfit on very noisy datasets
- Black-box compared to linear models

Model Comparison

The following table summarizes the key practical considerations when choosing among these six algorithms. The "best" model depends on the dataset size, dimensionality, interpretability requirements, and computational constraints of your specific problem.

Model	Training Speed	Prediction Speed	Interpretability	Handles Non-Linear	Scaling Required
Logistic Regression	Very Fast	Very Fast	High	No	Yes
Linear SVM	Medium	Fast	Medium	No (linear)	Yes
K Nearest Neighbours	Instant (lazy)	Slow	High	Yes	Yes
Naive Bayes	Very Fast	Very Fast	Medium	Yes	Depends
Decision Tree	Fast	Fast	Very High	Yes	No
Random Forest	Medium-Slow	Medium	Low	Yes	No

Practical Recommendations

> ***Start simple.*** *Begin with Logistic Regression or Naive Bayes as baselines. These are fast to train and provide interpretable results that help you understand your data.*

> ***When interpretability matters,*** *use a Decision Tree. Its hierarchical rule structure can be visualized and explained to non-technical stakeholders.*

> ***When performance is the priority,*** *try Random Forest. It typically delivers strong results with minimal tuning and is robust across many problem types.*

> ***For high-dimensional text data,*** *Naive Bayes and Linear SVM are traditional favourites that often outperform much more complex methods.*

Ultimately, the most reliable approach is to evaluate multiple models using cross-validation and select based on empirical performance on your specific task, rather than relying solely on theoretical properties.